Índice

Objetivos de la sesión
Conocimientos y materiales necesarios
1. Programación en C
  1.1. Primer programa
  1.2. Invocando funciones de biblioteca
  1.3. Salida por pantalla
  1.4. Punteros
  1.5. Estructuras
  1.6. Conversiones de tipos
Archivos de la práctica
Ejercicios
Apéndice A: Automatizando la generación del ejecutable

Introducción a la programación en C

Área de Arquitectura y Tecnología de Computadores – Versión 1.4.12002, 13/09/2023

Objetivos de la sesión

En esta sesión se introduce a la programación en C. Los principales objetivos de esta sesión son:

- Introducir los conceptos básicos del lenguaje C.
- Conocer el funcionamiento de los punteros en C.

Para conseguir estos objetivos se realizarán pequeños programas escritos en lenguaje C.

Conocimientos y materiales necesarios

Para poder realizar esta sesión, el alumno debe:

- Disponer de una máquina, física o virtual, con un sistema operativo Ubuntu Server 16.04.6 LTS.
- Durante la sesión se plantearán una serie de preguntas que puedes responder en el correspondiente cuestionario en el campus virtual. Puedes abrir el cuestionario en otra pestaña del navegador pinchando en el enlace mientras mantienes pulsada la tecla `Ctrl`.

---

1. Programación en C

El lenguaje de programación que se utilizará a lo largo de la asignatura será el lenguaje C. El lenguaje C es el lenguaje de programación de sistemas por excelencia y se encuentra íntimamente relacionado con los sistemas operativos UNIX. En esta sección repasaremos algunos conceptos sobre C y veremos algunos nuevos como los punteros.

- Arranca la máquina virtual y entra en sesión.
- Cambia al directorio de tu repositorio y sincronízalo con el repositorio de Bitbucket para actualizar los cambios que se hubiesen realizado con anterioridad.

```
$> git pull
```

- Crea un subdirectorio de nombre `sesion1-1` en el subdirectorio que contiene el repositorio de *git* `2ac`.

```
$> mkdir sesion1-1
```

- Entra en el nuevo directorio.

Deberás crear todos los programas que desarrolles a partir de ahora en el subdirectorio que acabas de crear.

1.1. Primer programa

Vamos a editar nuestro primer programa en C y ejecutarlo para ilustrar cuál es el procedimiento que debe seguirse en el desarrollo de programas.

- Edita el fichero `1-1hello.c` para que tenga el contenido que se muestra a continuación.

```c
#include <stdio.h>

int main()
{
    printf("Hello, World!\n");
    return 0;
}
```

En esta asignatura se adopta el convenio de programar en inglés. Cualquier graduado en informática debe ser capaz de leer y escribir código escrito en inglés, lo que le permitirá integrarse en proyectos internacionales.

Para generar el ejecutable es necesario realizar dos pasos: compilar el fuente y enlazar el ejecutable.

- Compilación: consiste en convertir los ficheros fuentes en lenguaje C a su equivalente código objeto (código máquina). La compilación se realiza invocando al compilador pasándole la opción `-c` junto con los ficheros fuente a compilar. En este caso:

```
$> gcc -c 1-1hello.c
$> ls *.o
1-1hello.o
```

- Enlazado: consiste en la generación del ejecutable a partir de los códigos objeto y las bibliotecas necesarias. Para realizar el enlazado se invoca al mismo compilador, que este caso actúa de enlazador, pasándole los ficheros con el código objeto. Opcionalmente, se le puede indicar el nombre del ejecutable deseado utilizando la opción `-o`. En otro caso, el compilador siempre generará un ejecutable de nombre `a.out`. Habría que indicarle también al compilador las bibliotecas con las que enlazar, pero en este caso no es necesario, pues no se necesitan bibliotecas adicionales. Por defecto el compilador enlaza el ejecutable con la biblioteca estándar de C, que contiene las funciones definidas por el estándar tales como `printf`.

```
$> gcc 1-1hello.o -o 1-1hello
$> ls -l 1-1hello
-rwxr-xr-x 1 student student 7155 2011-06-10 16:26 1-1hello
```

Como ves, se genera un fichero ejecutable de nombre `1-1hello`. Puedes invocar el ejecutable indicando la ruta en la que se encuentra, ya que por defecto el directorio de trabajo no está dentro de los directorios que consulta la interfaz de comandos para la ejecución de comandos y programas.

- Invoca el programa especificando únicamente su nombre. Verás que te aparece un mensaje indicativo de que no se encontró el comando.

```
$> 1-1hello
1-1hello: orden no encontrada
```

- Invoca el programa especificando ahora la ruta donde se encuentra, en este caso en el directorio actual. De esta forma, se le indica a la interfaz de comandos dónde debe buscar el comando.

```
$> ./1-1hello
Hello, World!
```

- Borra los ficheros `1-1hello.o` y `1-1hello`. Usa el comando `rm`.

Recuerda especificar la ruta cuando ejecutes programas creados durante las prácticas.

Por último, vamos a incorporar el fichero recién creado al repositorio *git*.

- Añade el fichero que acabas de crear al control de versiones.

```
$> git add 1-1hello.c
```

- Confirma los cambios.

```
$> git commit
```

1.2. Invocando funciones de biblioteca

Cuando un programador desarrolla programas en C, y en general en cualquier lenguaje de programación, utiliza funciones que se repiten entre programa y programa. Tal es el caso de las funciones que muestran información en pantalla, solicitan memoria de forma dinámica, trabajan con cadenas, etc. No tiene sentido que el programador tenga que escribir estas funciones una y otra vez, por lo que el lenguaje define un mecanismo para reutilizar estas funciones.

Estas funciones comunes pueden implementarse en ficheros separados que serán enlazados con el programa a desarrollar en la fase de enlazado. Estos ficheros pueden ser ficheros objetos como hemos visto o bibliotecas de funciones. No obstante, el compilador necesita conocer estas funciones que el programa utiliza pero que no define, para lo cual se utilizan ficheros de cabecera. Estos ficheros contienen declaraciones de funciones, es decir sus prototipos, que únicamente le indican al compilador cómo se llaman las funciones, cuántos parámetros reciben y de qué tipos.

En la fase de enlazado hay que especificarle al enlazador cuáles son los ficheros objetos o bibliotecas donde se encuentran estas funciones. La figura siguiente ilustra todo este proceso.

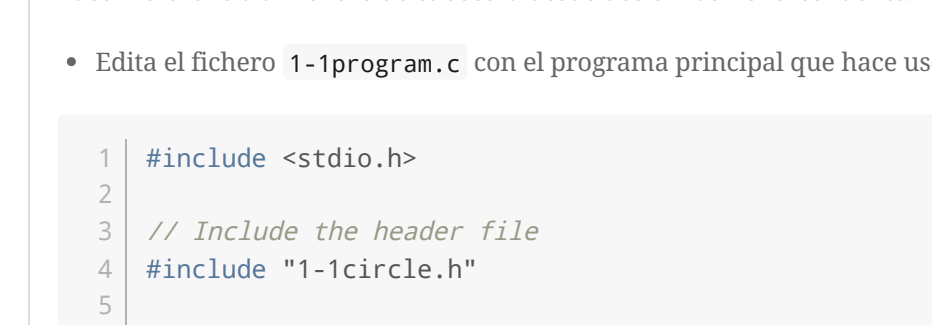

Figura 1. Ciclo de desarrollo de programas en C

Vamos a verlo con un ejemplo. Crearemos una función `circleArea` que devuelva el área de un círculo a partir de su radio. Esta función sería reutilizable por más programas, por lo que se definirá en un fichero distinto a donde definimos el programa principal.

- Edita el fichero `1-1circle.c` para que tenga el siguiente contenido:

```c
#include "1-1circle.h"

double circleArea(double radius)
{
    const double PI = 3.1415;
    return PI * radius * radius;
}
```

- Edita el fichero de cabecera `1-1circle.h` donde se definirá el prototipo de la función `circleArea`.

```c
// Always use this statement in header files
#pragma once

// Function prototype
double circleArea(double radius);
```

La línea `#pragma once` hace que el compilador añada a la compilación el código del fichero de cabecera una sola vez. Debe hacerse siempre para evitar errores de compilación por duplicidad de identificadores al hacer referencia al fichero de cabecera desde dos o más ficheros fuente.

- Edita el fichero `1-1program.c` con el programa principal que hace uso de la función círculo:

```c
#include <stdio.h>

// Include the header file
#include "1-1circle.h"

int main()
{
    double radius = 3.0;
    double area = circleArea(radius);
    printf("Circle area (radius=%f) is %f\n", radius, area);
    return 0;
}
```

- Compila ambos ficheros para obtener los ficheros objeto.

```
$> gcc -c 1-1circle.c 1-1program.c
```

- Enlaza el programa indicando el fichero objeto donde está la función `circleArea`.

```
$> gcc 1-1circle.o 1-1program.o -o 1-1program
```

- Ejecuta el programa resultante e incorpora los tres ficheros fuente a tu repositorio (`git add` y `git commit`).

El estándar que describe el lenguaje C define adicionalmente una biblioteca de funciones que deben incluir todos los compiladores denominada la biblioteca estándar de C. Esta biblioteca incluye el código de las funciones definidas en el estándar de C.

Un ejemplo de función incluida en la biblioteca estándar de C es la función `printf` antes citada. Puedes consultar el manual de la función para ver cómo se invoca y qué fichero de cabecera es necesario incluir.

- Abre el manual de la función `printf`.

```
$> man 3 printf
```

- ¿Qué fichero de cabecera es necesario incluir? Responde en el cuestionario: pregunta 1.

El enlazador por defecto enlaza todos los programas con la biblioteca estándar de C, por lo que solo debes preocuparte de incluir el fichero de cabecera correspondiente.

Sin embargo, hay algunas funciones que se definen en otras bibliotecas. Tal es el caso de la función `sqrt`, que calcula la raíz cuadrada de un número real.

- Consulta el manual de la función `sqrt`.

```
$> man 3 sqrt
```

- ¿Qué fichero de cabecera es necesario incluir? Responde en el cuestionario: pregunta 2.
- ¿Es necesario indicarle alguna opción al enlazador? Responde en el cuestionario: pregunta 3.
- Edita el fichero `1-1root.c`.

```c
#include <stdio.h>
// TODO: Add the required #include directive

int main()
{
    double num = 9.0;
    double root = sqrt(num);
    printf("The square root of %f is %f\n", num, root);
    return 0;
}
```

- Compila el programa.
- Enlaza el programa, verás que te aparece un error pues la función `sqrt` no está definida. Recuerda que solo has incluido el fichero de cabecera, que contiene únicamente el prototipo de la función, no su código.

```
$> gcc 1-1root.o -o 1-1root
1-1root.o: In function 'main':
1-1root.c:(.text+0x2d): undefined reference to 'sqrt'
collect2: ld returned 1 exit status
```

- Enlaza ahora el programa incluyendo en la línea de comandos la opción que te indicaba la página del manual de `sqrt`.
- Ejecuta el programa e incorpora el fuente al repositorio *git*.

Es de vital importancia que sepas diferenciar los errores de compilación de los errores de enlazado.

1.3. Salida por pantalla

Para mostrar información por pantalla utilizaremos la función `printf`, que se encuentra declarada en el fichero de cabecera `stdio.h`. Para imprimir por pantalla una cadena de texto se podría utilizar la siguiente sentencia:

```c
printf("This string is shown on the screen");
```

Esta función recibe un número variable de parámetros, de los cuales el primero es una cadena de formato que puede contener:

- Texto. Este texto se mostrará en la pantalla tal cual.
- Secuencias de escape. Comienzan por el carácter `\` y van seguidas por otro carácter. Las secuencias `\n` y `\t` representan un salto de línea y una tabulación respectivamente.
- Especificadores de formato. Se utilizan para mostrar el valor de variables. Comienzan por el carácter `%`:
  - `%c`: carácter (tipo `char`).
  - `%d`: entero decimal con signo (tipo `int`).
  - `%f`: real (tipo `float` o `double`).
  - `%s`: cadena de caracteres (tipo `char *`).
  - `%x`: entero en formato hexadecimal.
  - `%p`: puntero.

El resto de parámetros de la función son las variables a mostrar que deben aparecer en el mismo orden que aparecen los especificadores de formato.

- Edita el fichero `1-1output.c` con el contenido que se muestra a continuación.

```c
#include <stdio.h>

int main()
{
    int i = 3;
    float f = 2.4;
    char * s = "Content";

    // TODO: Print the content of i

    // TODO: Print the content of f

    // TODO: Print the content of the string


    return 0;
}
```

- Completa las líneas comentadas que faltan de tal forma que al ejecutar el programa lo que se muestre por pantalla sea lo siguiente.

```
i content: 3
f content: 2.400000
s content: Content
```

- Compila y enlaza el programa.
- Ejecuta el programa.

```
$> ./1-1output
```

- Incorpora el fichero fuente `1-1output.c` al repositorio *git*.

En general, cuando el tamaño de los enteros no es relevante, estos suelen definirse como se ha hecho en el ejemplo anterior, empleando el tipo `int`. En el caso de la máquina empleada el tamaño para el tipo `int` es de 32 bits, es decir, 4 bytes. Sin embargo, cuando el tamaño del entero es relevante suelen emplearse unos tipos alternativos que de forma explícita indican su tamaño en bits: `int8_t`, `int16_t`, `int32_t` e `int64_t`. Para poder emplearlos debe incluirse el archivo de cabecera `inttypes.h`. Por lo tanto, el tipo `int32_t` es equivalente al tipo `int` en esta máquina.

Aunque el estándar define unos especificadores de formato propios para los tipos anteriores, por simplicidad seguiremos usando los habituales. En el caso de nuestra máquina, con los tipos `int8_t`, `int16_t` e `int32_t` se puede emplear el especificador de formato `"%d`, mientras que con el tipo `int64_t` el especificador de formato a usar es `%lld`.

1.4. Punteros

Los punteros y su aritmética es con mucho la parte más complicada del lenguaje C, pero también la que le da la capacidad de manejar la memoria a un nivel muy bajo, básica para programación de sistemas. Un puntero es una variable que guarda una dirección de memoria, o lo que es lo mismo, que apunta a un dato en memoria, de ahí su nombre. Para declarar un puntero es necesario indicar el tipo de datos al que apunta junto con el carácter `*`. Algunas definiciones de punteros son las siguientes.

```c
char * pChar;  // Refers to a piece of data in memory 1-byte wide
int32_t * pInt;  // Refers to a 32-bits integer in memory
double * pDouble1, * pDouble2;  // Refer to float numbers with double
                                // precision (8-bytes wide)
```

Un puntero definido de esta forma no puede ser directamente utilizado, ya que apunta a una zona de memoria desconocida. En lugar de ello, hay que hacer que el puntero apunte a una zona de memoria válida para poder leer y escribir en las posiciones de memoria a las que apunta. Esto puede hacerse de dos formas: haciendo que el puntero apunte a la zona de memoria de una variable del programa (utilizando el operador `&` para obtener la dirección de una variable), o haciendo que el puntero apunte a una zona de memoria reservada de forma dinámica, habitualmente en el montículo o *heap*.

El montículo es una zona de memoria del programa que se reserva de forma dinámica.

```c
#include <stdlib.h> // Required by malloc()
#include <inttypes.h>

int main()
{
    int32_t x;
    int32_t * pInt;
    char * pChar;

    x = 3;
    pInt = &x;
    pChar = (char *)malloc(20 * sizeof(char));
    free(pChar);
    return 0;
}
```

Observa en el ejemplo anterior cómo se reserva memoria de forma dinámica en el montículo con la función `malloc`. El operador `sizeof` devuelve el tamaño en bytes de un tipo. Además, es necesario hacer una conversión (*casting*) al asignar la dirección al puntero, ya que la función retorna `void *`, que quiere decir «puntero a un tipo indeterminado».

A diferencia de lenguajes que incorporan recolector de basura como Java, en C es necesario liberar la memoria reservada de forma explícita con la función `free` cuando ya no se use.

Para utilizar los punteros se usa el operador indirección `*`. Cuando este operador precede al nombre de una variable puntero, significa que se hace referencia al valor de la memoria a la que el puntero apunta. Si a continuación del código anterior se añade la siguiente sentencia justo antes del `return`, la salida por pantalla sería 3 3.

```c
printf("%d %d", x, *pInt);
```

Un uso muy común de los punteros es en el paso de parámetros a una función. En C los parámetros se pasan por valor[1]. Esto quiere decir que si se utiliza una variable como parámetro de una función, se copia su valor a una nueva variable dentro de la función. Si se cambia este valor dentro de la función, el valor de la variable que está fuera de la función no cambiará.

Aunque una función siempre recibe los parámetros por valor (recibe una copia del valor de la variable), en la práctica se puede conseguir el paso por referencia si se le pasa la dirección de la variable, es decir, un puntero. En este último caso la función recibe una copia del puntero, con lo que es posible modificar los datos a los que apunta el puntero usando el operador de indirección pero no la dirección (el puntero en sí).

El siguiente código muestra un ejemplo de paso de una variable a un procedimiento por valor.

```c
#include <stdio.h>

void f(double d)
{
    d = 4;
}

int main(int argc, char* argv[])
{
    double d;
    d = 3;

    f(d);
    printf("%f\n", d);
    return 0;
}
```

- Escribe el programa `1-1pointer1.c` a partir del listado anterior y comprueba cuál es el valor mostrado por pantalla.
- Ahora modifica el programa, llamándolo `1-1pointer2.c`, para que la función `f` en lugar de recibir un `double` reciba un puntero a un `double`. Además, tendrás que modificar la asignación para modificar el contenido de la dirección apuntada por el puntero (utiliza el operador de indirección). Por último, deberás modificar la llamada a la función para que reciba la dirección de la variable `d`. Comprueba que el programa imprime ahora el valor `4`. Si no es así, consulta a tu profesor.
- Incorpora ambos ficheros fuente al repositorio *git*.

1.5. Estructuras

Las estructuras son agrupaciones de tipos básicos en memoria, de tal forma que se ubican en memoria varios elementos de tipos básicos uno a continuación de otro. El siguiente programa muestra cómo definir y utilizar una estructura denominada `Person` que incluye una cadena de texto, un entero y un real de precisión doble.

- Edita el fichero `1-1person.c` con el contenido que se muestra a continuación.

```c
#include <stdio.h>
#include <string.h>

struct _Person
{
    char name[30];
    int heightcm;
    double weightkg;
};
// This abbreviates the type name
typedef struct _Person Person;

int main(int argc, char* argv[])
{
    Person Peter;
    strcpy(Peter.name, "Peter");
    Peter.heightcm = 175;

    // TODO: Assign the weight

    // TODO: Show the information of the Peter data structure on the screen

    return 0;
}
```

- Completa las líneas comentadas que faltan de tal forma que al ejecutar el programa se muestre por pantalla lo siguiente.

```
Peter's height: 175 cm; Peter's weight: 78.7 kg
```

- Genera el ejecutable y ejecútalo.
- Incorpora el fichero al repositorio *git*.

También es posible crear punteros a estructuras. Volviendo a la estructura `Person` que creamos anteriormente, podemos definir un nuevo tipo puntero a dicha estructura.

```c
// Type name abbreviations
typedef struct _Person Person;
typedef Person* PPerson;

int main(int argc, char* argv[])
{
    Person Javier;
    PPerson pJavier;

    // Memory location of variable Javier is assigned to the pointer
    pJavier = &Javier;
    Javier.heightcm = 180;
    Javier.weightkg = 84.0;
    pJavier->weightkg = 83.2;
    return 0;
}
```

Fíjate cómo para acceder a los campos de una estructura a través del puntero se debe hacer uso del operador `->`. Por ejemplo, `pJavier->weightkg` es equivalente a `(*pJavier).weightkg`, es decir, acceder a donde apunta `pJavier` (que es una estructura de tipo `Person`) y luego a su campo `weightkg`.

- Modifica el fichero `1-1person.c` para añadir el tipo puntero a persona `PPerson` tal como se indicó anteriormente.
- Incorpora los cambios sobre el fichero al repositorio *git*.

1.6. Conversiones de tipos

El lenguaje C permite conversiones entre diferentes tipos básicos. Las conversiones pueden realizarse de forma implícita o explícita.

En el primer caso, no es necesaria ninguna instrucción por parte del programador, sino que es el propio lenguaje el que define las conversiones más adecuadas a la hora de evaluar expresiones. Estas conversiones generan avisos en algunos compiladores en el momento de la compilación si implican pérdida de información.

En cambio, las conversiones explícitas se producen a instancia del programador. Se utilizan para convertir entre tipos que no tienen conversión (simplemente se le insta al compilador a interpretar el valor como el nuevo tipo) y para evitar los avisos del compilador cuando se producen conversiones con pérdida de información.

Vamos a verlo con un ejemplo.

- Edita el fichero `1-1implicit.c` con el contenido que se muestra a continuación.

```c
#include <stdio.h>

int main() {
    double div = 3.75;
    int num = 6;
    int result = num / div;
    printf("Variable result is %d, but the division was %f\n", result, num /
div);
    return 0;
}
```

- Compila y ejecuta el programa. Verás que se compila sin mostrar ningún aviso, pero el resultado de la división no coincide con el mostrado.

  En las últimas versiones de gcc no se muestran avisos cuando se producen conversiones implícitas con pérdida de información. Para mostrar este tipo de avisos puede utilizarse la opción `-Wconversion`.

- Compila de nuevo el programa pero indicando la orden `-Wconversion`.
- Verás que ahora aparece un aviso indicando que el valor puede variar al hacer la conversión.
- Intenta comprender qué conversiones se producen al evaluar la expresión del ejemplo. Si tienes dudas pregúntale a tu profesor.
- Para evitar el aviso, escribe justo delante de la expresión entre paréntesis el tipo al que se va a convertir para realizar un *casting* (conversión explícita). De esta forma se le indica al compilador que sabemos que se produce esa conversión a pesar de perder información.
- Compila de nuevo con la opción `-Wconversion` para comprobar que el aviso ya no se produce.

La mayor potencia de la conversión de tipos en C viene cuando se combina con punteros. Esto permite interpretar posiciones de la memoria como si se tratasen de cualquier tipo de datos.

Vamos a verlo con un ejemplo que nos va a permitir comprobar la *endianness* de la máquina (orden en el que se almacenan los bytes en memoria para un dato de más de un byte).

- Edita el fichero `1-1explicit.c` con el contenido que se muestra a continuación.

```c
#include <stdio.h>
#include <inttypes.h>

int main() {
    int32_t x = 1;

    const char *p = (const char*)&x;

    if (*p) {
        printf("Little-endian\n");
    }
    else {
        printf("Big-endian\n");
    }

    return 0;
}
```

- Fíjate cómo el puntero se hace apuntar a la dirección más baja del entero de 32 bits, lo que permite interpretar este byte como un carácter.
- Si tienes dudas pregúntale a tu profesor.
- Incorpora los ficheros `1-1implicit.c` y `1-1explicit.c` al repositorio *git*.

Archivos de la práctica

Debes guardar todos los ficheros sobre los que has trabajado durante la práctica. Si has incorporado los cambios realizados a tu repositorio, basta con sincronizar tu repositorio con el almacenado en Bitbucket.

- Sincroniza tu repositorio en Bitbucket con el repositorio local. El siguiente comando sube a Bitbucket los cambios que has incorporado a tu repositorio local.

```
$> git push
```

- Comprueba en Bitbucket que están todos los ficheros.

Ejercicios

Se proponen los siguientes ejercicios adicionales:

- Haz un programa de nombre `1-1add.c` que defina una función `add` que reciba por parámetro un vector de enteros y su tamaño. Dicha función debe devolver como resultado la suma de todos los elementos del vector. La función `main` de dicho programa será la siguiente.

```c
#include <stdio.h>

#define NUM_ELEMENTS 7

int main()
{
    int vector[NUM_ELEMENTS] = { 2, 5, -2, 9, 12, -4, 3 };
    int result;

    result = add(vector, NUM_ELEMENTS);
    printf("The addition is: %d\n", result);
    return 0;
}
```

  Incorpora `1-1add.c` al repositorio *git*.

- Haz un programa de nombre `1-1copy.c` que copie cadenas de texto a través de la función `copy` que tendrá el siguiente prototipo.

```c
int copy(char * source, char * destination, unsigned int lengthDestination);
```

  En el primer parámetro recibe la cadena a copiar. Recuerda que las cadenas en C terminan con `0`. El segundo parámetro es un puntero a la zona de memoria donde copiar la cadena, mientras que el tercero indica el tamaño de esta zona, para evitar que la función escriba fuera de esta zona.

  Incorpora `1-1copy.c` al repositorio *git*.

Apéndice A: Automatizando la generación del ejecutable

Los sistemas UNIX suelen incluir una herramienta, denominada `make`, que permite automatizar la generación de ejecutables. Para ello, utiliza un fichero que habitualmente se llama `Makefile` y que contiene una serie de reglas que indican cómo debe generarse el ejecutable teniendo en cuenta qué ficheros deben regenerarse cada vez que cambien otros. Cuando un fichero A debe regenerarse cuando cambie un fichero B, se dice que B es una dependencia de A.

La gran ventaja de la herramienta `make` es que automáticamente comprueba si los ficheros que se especifican como dependencias han sido modificados, y genera solo los ficheros que dependan de dichos ficheros modificados, con lo que se ahorra tiempo al generar los programas.

Las reglas de un fichero `Makefile` asocian un objetivo con las dependencias y los comandos que son necesarios ejecutar para, a partir de las dependencias, obtener el objetivo. Son de la forma:

```
objetivo : dependencias
<Tabulador> comandos
```

Antes de indicar el comando que crea el objetivo a partir de la lista de dependencias hay que utilizar un carácter de tabulación. A continuación se muestra un ejemplo de `Makefile` que puede utilizarse para compilar y enlazar el programa `1-1program` visto anteriormente.

```makefile
# Rule for generating the main program
1-1program : 1-1program.o 1-1circle.o
    gcc 1-1program.o 1-1circle.o -o 1-1program

# Compile source files
1-1program.o : 1-1program.c 1-1circle.h
    gcc -c 1-1program.c

1-1circle.o : 1-1circle.c 1-1circle.h
    gcc -c 1-1circle.c

# Clean object files
clean :
    rm -f 1-1program.o 1-1circle.o
```

- Edita el fichero `Makefile` en el mismo directorio que los ficheros `1-1program.c`, `1-1circle.c` y `1-1circle.h` de forma que contega el listado mostrado.

  Ten cuidado al copiar y pegar en la consola el listado del fichero, pues deberás sustituir los espacios por tabulaciones.

- Para que veas cómo compila make, borra todos los ficheros objeto con `rm *.o` y borra también `1-1program`.
- Genera el ejecutable invocando al comando `make`. Puedes hacerlo indicando el nombre del ejecutable a generar o bien sin parámetros, ya que la regla que genera el ejecutable es la primera del fichero `Makefile`.

```
$> make
gcc -c 1-1program.c
gcc -c 1-1circle.c
gcc 1-1program.o 1-1circle.o -o 1-1program
```

- Intenta volver a generar el ejecutable. El comando `make` comprobará que las dependencias del objetivo `1-1program` no han cambiado, por lo que inferirá que el ejecutable está actualizado.

```
$> make
make: '1-1program' está actualizado.
```

- Elimina los ficheros temporales generados invocando la regla `clean` del `Makefile`.

```
$> make clean
rm -f 1-1program.o 1-1circle.o
```

- Incorpora el fichero `Makefile` al repositorio *git*.

Puedes utilizar este fichero `Makefile` como patrón para generar el resto correspondientes a los programas que desarrollarás a lo largo de las prácticas. Debes tener cuidado de modificar apropiadamente los nombres de fichero que aparecen en el `Makefile` para adecuarlos al programa a generar.

---

[1] En C++ las funciones pueden recibir referencias a variables (por ejemplo, el tipo de un parámetro puede ser `int&`, es decir, una referencia a entero). Sin embargo, C no tiene referencias.